Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Introduction to sorting

Today
Welcome to your next lesson: Introduction to sorting.

I'm setting up your personalized lesson, using what you've already learned and what comes next for you.

Step by step, you’re moving forward.

Think about your phone’s contacts list 📱

If it’s sorted A→Z, finding “Ron Hensley” is easy: you jump near the R’s and scroll a tiny bit.
If it’s random, you have to scan almost every name each time.

So sorting is about putting data in a clear order so things like searching, reports, and decisions become much simpler and faster.

Quick check for you, Ron:

In your own life, where do you feel sorting helps you more?

Contacts list
Game leaderboards
Email inbox
Something else?

Just name one example that feels most real to you, and we’ll use that as your running example next.

contacts

Nice, we’ll use contacts as our main example 📱

Imagine your contacts are totally unsorted. To find “Ron Hensley”:

You start at the top
Scan name… name… name…
Maybe you miss it and have to scroll back 😅

If the contacts are sorted A→Z:

You jump straight to the R section
Scroll a little
Done ☑

So sorting is what lets your phone avoid scanning every single contact each time.

Quick question:

If your contacts were sorted, which of these becomes easier?

A) Finding one specific contact

B) Seeing who has the earliest name alphabetically

C) Both A and B

You can just answer with A, B, or C.

c

Exactly 👍 C is right.

Once contacts are sorted:

Finding one person is easier.
Answering questions like “Who’s first alphabetically?” is also easier.
Now I want to show you one simple way a computer can sort:

It keeps looking at neighbour pairs and, whenever two neighbours are in the wrong order, it swaps them. Over and over, the list slowly “bubbles” into order.

Tiny example with last names:

["Smith", "Adams", "Brown"]

Step by step idea:

Compare neighbors "Smith" and "Adams" → S comes after A, so swap.
Now list is ["Adams", "Smith", "Brown"]
Compare "Smith" and "Brown" → S comes after B, so swap.
Now list is ["Adams", "Brown", "Smith"] → sorted.

So the rule is:

Look at two neighbors; if they’re out of order, trade places. Repeat.

Does this “keep swapping neighbors until everything is in order” idea make sense so far, or should I walk through one more small example with different names?

I understand it

Great 😊

Let’s look at a different kind of idea for sorting your contacts.

Instead of swapping neighbours, imagine this strategy:

Look through the whole list and find the smallest (earliest alphabetically) name.
Move that name to the front.
Then, from the remaining names, again find the smallest and put it in the next spot.
Repeat until everything is placed.

Tiny example:

Start:

["Smith", "Adams", "Brown"]

Smallest is "Adams" → put it first → ["Adams", "Smith", "Brown"]
From the rest ["Smith", "Brown"], smallest is "Brown" → next → ["Adams", "Brown", "Smith"]
Last one "Smith" just stays.

So the rule is:

Repeatedly pick the smallest remaining item and put it into its correct place.

Between the two ideas, which feels more natural to you for contacts:

swapping neighbours over and over, or repeatedly picking the smallest remaining name and placing it?

bubble

What do you think? Pick the answer that feels most true to you 👇

You have 1,000 contacts on your phone. You often search for a specific person by name. What's the main benefit of sorting the contacts A→Z once before doing many searches?

A.

Each single search becomes faster and needs to check fewer contacts.

B.

The phone uses less battery overall, even if searches are the same speed.

C.

Sorting makes no difference; the phone still checks all 1,000 contacts every time.

D.

Sorting only helps if you never add new contacts again.

Nice choice, Ron 🎯 You're exactly right.

If you sort once and then search many times:

Each search can skip huge chunks of the list (using something like binary search later on),
So it checks far fewer contacts per search, making every lookup faster.

The battery / "never add contacts again" parts aren't the core idea — the real win is: pay a one-time sorting cost, then all future searches get cheaper.

You’ve now seen:

Why sorting your contacts helps searching,
And two big-picture strategies: swap neighbours vs pick the smallest each time.

Want to quickly recap the two strategies in your own words, or go straight to the next lesson?

bubble sort compars 2 neighbors and if it of order swaps then copares to ext then next until its sorted. the second sort goes throuht the linst traking the lowest value, places it at the begiining then repeats starting at 2nd slot then again at 3r slot tc till sorte

Love how you explained both, Ron 👍

Neighbour-swapping idea (what you called bubble):
Compare 2 neighbours, if they’re out of order, swap them, then keep moving along the list again and again until nothing is out of order.

Smallest-first idea:
Scan the list to track the lowest value, put it at the front, then repeat starting from the 2nd position, then 3rd, and so on until everything is sorted.

You’ve nailed exactly what this lesson wanted:

Why sorting helps (faster searches, easier questions about the data),
Two high-level strategies to sort (swap neighbours vs pick-the-smallest).
You’re all set for the next lesson 🎉
When you’re ready, hit Next in the Learn tab, and we’ll build on this. If anything about sorting feels fuzzy later, just ask and I’ll zoom back in with you.